Tool / Function Schema Reference — Ultron Agent

This document defines every tool the fine-tuned model is trained to call, in the OpenAI-compatible function-calling format. The model itself never executes a tool — it only emits a structured call; the orchestration layer validates, authorizes, and executes it (see `RESPONSIBLE_USE_POLICY.md` and `API_INTEGRATION_GUIDE.md`).

> Adjust names/parameters to match your actual implementation before the demo — this is the reference structure judges will expect, not a fixed list you must keep verbatim.

1. `start_engagement`

Registers a scoped, authorized engagement before any other tool may be called.

```
{
  "name": "start_engagement",
  "description": "Register a new authorized penetration-testing engagement and scope before any scan or exploit tool run.",
  "parameters": {
    "type": "object",
    "properties": {
      "engagement_id": { "type": "string", "description": "Unique identifier for this engagement" },
      "authorized_targets": { "type": "array", "items": { "type": "string" }, "description": "IP ranges/hostnames explicitly in scope" },
      "authorization_reference": { "type": "string", "description": "Reference to signed rules-of-engagement / authorization document" }
    },
    "required": ["engagement_id", "authorized_targets", "authorization_reference"]
  }
}
```

2. `run_recon_scan`

Runs a non-destructive discovery/enumeration scan against in-scope targets only.

```
{
  "name": "run_recon_scan",
  "description": "Run a non-destructive host/service discovery scan against a target already registered in the current engagement scope.",
  "parameters": {
    "type": "object",
    "properties": {
      "engagement_id": { "type": "string" },
      "target": { "type": "string", "description": "Must be a member of the engagement's authorized_targets" },
      "scan_type": { "type": "string", "enum": ["host_discovery", "port_scan", "service_fingerprint"] }
    },
    "required": ["engagement_id", "target", "scan_type"]
  }
}
```

3. `lookup_vulnerability_reference`

Read-only lookup against public vulnerability reference data (e.g., CVE/CWE metadata) — no scanning or execution.

```
        {
          "name": "lookup_vulnerability_reference",
          "description": "Retrieve public reference metadata (description, severity, affected versions) for a known CVE or C
identifier.",
          "parameters": {
            "type": "object",
            "properties": {
              "identifier": { "type": "string", "description": "CVE or CWE identifier, e.g. CVE-2024-1234" }
            },
            "required": ["identifier"]
          }
        }
```

4. `request_human_confirmation`

Required gate before any state-changing or higher-impact action.

```
        {
          "name": "request_human_confirmation",
          "description": "Pause execution and require a human operator to explicitly approve a proposed next action before i
runs.",
          "parameters": {
            "type": "object",
            "properties": {
              "engagement_id": { "type": "string" },
              "proposed_action": { "type": "string", "description": "Plain-language description of the action awaiting appro
},
              "risk_level": { "type": "string", "enum": ["low", "medium", "high"] }
            },
            "required": ["engagement_id", "proposed_action", "risk_level"]
          }
        }
```

5. `log_finding`

Records a finding for the final report; read/write only to the engagement's own report object.

```
        {
          "name": "log_finding",
          "description": "Record a discovered issue (with evidence reference) to the current engagement's findings log.",
          "parameters": {
            "type": "object",
            "properties": {
              "engagement_id": { "type": "string" },
              "title": { "type": "string" },
              "severity": { "type": "string", "enum": ["info", "low", "medium", "high", "critical"] },
              "evidence_ref": { "type": "string", "description": "Pointer to stored scan/tool output supporting this finding
              "recommendation": { "type": "string" }
            },
            "required": ["engagement_id", "title", "severity", "evidence_ref"]
          }
        }
```

6. `generate_report`

Compiles all logged findings for the engagement into a client-ready report.

```
        {
          "name": "generate_report",
          "description": "Generate a formatted report from all findings logged for the given engagement.",
          "parameters": {
            "type": "object",
            "properties": {
              "engagement_id": { "type": "string" },
              "format": { "type": "string", "enum": ["markdown", "pdf", "docx"] }
            },
            "required": ["engagement_id", "format"]
          }
        }
```

Design Notes

- **Scope enforcement lives outside the model.** Every tool that touches a target takes `engagement_id` / `target`, and the orchestration layer — not the model — is responsible for rejecting any call against a target not in `authorized_targets`.
- **Higher-impact actions route through `request_human_confirmation` first.** The fine-tuning dataset was built to teach the model to *always* propose this gate before any action beyond read-only recon or lookups.
- **No exploit-execution tool is defined here.** This reference intentionally stops at discovery, reference lookup, human-gated approval, and reporting — actual exploit/attack execution tooling, if used in your build, should carry its own stricter authorization, logging, and kill-switch design reviewed separately from this document.